

Airline Fare Prediction

Dynamic Pricing Analytics using Power BI & Python



Prepared by: Divya Upadhyay

25/09/2026

GitHub Link: [[GitHub](#)]

Outline

- Executive
- Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary (Methodologies)

1. Data Sourcing & Attributes:

Collected historical domestic flight records across key Indian metropolitan routes, capturing carriers, journey dates, route paths, duration, layover stops, and ticket prices.

2. Python Preprocessing & Cleansing:

Resolved missing route/layover records, parsed string durations into total minutes, and normalized anomalous fare outliers.

3. Feature Engineering:

Extracted temporal attributes (day, month, weekday), binned departure/arrival times into 5 intervals, and mapped total layovers to numeric values.

4. Power BI Modeling & DAX:

Created DAX measures for fare variance, built KPI summaries with carrier distributions, and deployed AI Decomposition Trees to evaluate multi-variable volatility.

5. Predictive Regression Modeling:

Trained tree-based ensemble models (Random Forest, Decision Tree) to capture non-linear patterns, evaluating fit using R^2 score and MAE benchmarks.

GitHub Link: [\[GitHub\]](#)

Executive Summary (Results)

- **1. Dynamic Pricing & Perishable Inventory Economics:**
 - Unlike physical retail with carryover stock, airline seats represent **zero-salvage perishable inventory** where an unfilled cabin upon gate closure translates to permanent, unrecoverable revenue loss.
 - Carriers deploy algorithmic revenue management systems that exploit extreme **inelastic demand curves**, driving intra-day price swings between **40% and 150%** based on lead-time decay and remaining capacity tiers.
- **2. The Layover & Hub Transit Cost Paradox:**
 - Point-to-point non-stop flights consolidate takeoff and climb phases—the most fuel-intensive flight segments—alongside single-terminal ground and air traffic control handling fees.
 - Connecting itineraries introduce **multi-airport landing tariffs, passenger service fees, and hub congestion taxes**, disproving the assumption of layover discounts and driving multi-stop prices significantly above direct options.
- **3. Enterprise Procurement & Revenue Risk Mitigation:**
 - Corporate travel programs and aggregators experience substantial budget leakage due to opaque carrier price discrimination and variable fuel surcharge indexing.
 - Isolating non-linear fare inflection points establishes data-backed procurement thresholds, mitigating last-minute surge premiums and enhancing forward-booking cost optimization.
- **4. Quantitative Scope & Multi-Tier Analytical Framework:**
 - Analyzes an empirical dataset of **10,683 domestic flight records** spanning key metropolitan corridors to capture multidimensional pricing behavior across carriers, routes, and temporal shifts.
 - Combines automated Python feature engineering pipelines with interactive Power BI diagnostic modeling, delivering an operational architecture for predictive fare valuation and variance attribution.
- GitHub Link: [[GitHub](#)]

Introduction

- **1. Dynamic Pricing & Perishable Inventory:**
 - Commercial aviation relies on perishable seats where unsold inventory causes irreversible revenue loss, creating high price volatility.
 - Last-minute demand spikes, carrier yield management, and lead-time decay routinely swing fares between **40% and 150%** within hours.
- **2. The Layover & Hub Transit Paradox:**
 - Direct non-stop flights consolidate fuel burn and airport tariffs into a single operational cost structure.
 - Connecting itineraries accumulate landing fees, transit taxes, and hub operational costs, pushing multi-stop tickets significantly above non-stop rates.
- **3. Enterprise Procurement & Margin Protection:**
 - Lack of dynamic fare transparency leads to significant financial leakage for corporate travel desks and flight aggregators.
 - Predicting critical price inflection points identifies cost-effective booking windows and eliminates severe last-minute surge penalties.
- **4. Project Scope & Analytical Objectives:**
 - Synthesizes **10,683 domestic flight records** across high-traffic routes to evaluate non-linear pricing behavior.
 - Integrates Python-driven feature pipelines and Power BI intelligence to deliver an end-to-end predictive fare forecasting system.
- GitHub Link: [\[GitHub\]](#)
-

Section 1

Methodology

GitHub Link: [[GitHub](#)]



Data Collection – Airline Fare Prediction (1/3)

Data Ingestion: Raw Dataset Structure & Attributes

- The dataset ingestion pipeline is outlined in the next two slides.

- Sample extraction showing raw domestic flight records across key carriers.

GitHub Link: [\[GitHub\]](#)

```
data_train.head(10)
```

	Airline	Date_of_Journey	Source	Destination	Route	Dep_Time	Arrival_Time	Duration	Total_Stops	Additional_Info	Price
0	IndiGo	24/03/2019	Banglore	New Delhi	BLR → DEL	22:20	01:10 22 Mar	2h 50m	non-stop	No info	3897
1	Air India	1/05/2019	Kolkata	Banglore	CCU → IXR → BBI → BLR	05:50	13:15	7h 25m	2 stops	No info	7662
2	Jet Airways	9/06/2019	Delhi	Cochin	DEL → LKO → BOM → COK	09:25	04:25 10 Jun	19h	2 stops	No info	13882
3	IndiGo	12/05/2019	Kolkata	Banglore	CCU → NAG → BLR	18:05	23:30	5h 25m	1 stop	No info	6218
4	IndiGo	01/03/2019	Banglore	New Delhi	BLR → NAG → DEL	16:50	21:35	4h 45m	1 stop	No info	13302
5	SpiceJet	24/06/2019	Kolkata	Banglore	CCU → BLR	09:00	11:25	2h 25m	non-stop	No info	3873
6	Jet Airways	12/03/2019	Banglore	New Delhi	BLR → BOM → DEL	18:55	10:25 13 Mar	15h 30m	1 stop	In-flight meal not included	11087
7	Jet Airways	01/03/2019	Banglore	New Delhi	BLR → BOM → DEL	08:00	05:05 02 Mar	21h 5m	1 stop	No info	22270
8	Jet Airways	12/03/2019	Banglore	New Delhi	BLR → BOM → DEL	08:55	10:25 13 Mar	25h 30m	1 stop	In-flight meal not included	11087
9	Multiple carriers	27/05/2019	Delhi	Cochin	DEL → BOM → COK	11:25	19:15	7h 50m	1 stop	No info	8625

Data Collection – Airline Fare Prediction (2/3)

Data Schema, Feature Types & Missing Value Diagnostics

1.Dataset Scale & Shape:

Total **10,683 rows** and **11 feature columns** loaded for processing.

2.Feature Type Diagnostics:

Categorical/Strings: Attributes like Date_of_Journey, Route, Dep_Time, and Duration are stored as object types, requiring parsing into structured datetimes and numeric integers.

Target Column: Price is loaded as an integer (int64), representing domestic ticket costs in INR.

3.Missing Value Inspection:

Minimal null values detected in Route and Total_Stops (only 1 record), which are filtered out to maintain data integrity.

GitHub Link: [\[GitHub\]](#)

```
data_train.info() # first we will start w

<class 'pandas.DataFrame'>
RangeIndex: 10683 entries, 0 to 10682
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Airline                10683 non-null  str
1   Date_of_Journey        10683 non-null  str
2   Source                  10683 non-null  str
3   Destination             10683 non-null  str
4   Route                   10682 non-null  str
5   Dep_Time                10683 non-null  str
6   Arrival_Time            10683 non-null  str
7   Duration                10683 non-null  str
8   Total_Stops             10682 non-null  str
9   Additional_Info         10683 non-null  str
10  Price                   10683 non-null  int64
dtypes: int64(1), str(10)
memory usage: 1.8 MB
```

Data Collection – Airline Fare Prediction (3/3)

Missing Value Treatment & Carrier Distribution Baseline

1. Missing Value Treatment:

Detected only **1 null row** in Route & Total_Stops.
Cleaned using `data_train.dropna(inplace=True)`.

2. Data Retention:

Retained **10,682 clean rows** (~99.99% data intact).
Preserved original corridor pricing distribution.

3. Carrier Baseline:

Led by **Jet Airways, IndiGo, and Air India**.
Covers both budget carriers and legacy airlines.

GitHub Link: [\[GitHub\]](#)

```
data_train.isnull().sum()  
# data_train.isnull().sum(axis = 0)  
# so here in [sum(axis = 0)] by default here axis = 0
```

Airline	0
Date_of_Journey	0
Source	0
Destination	0
Route	1
Dep_Time	0
Arrival_Time	0
Duration	0
Total_Stops	1
Additional_Info	0
Price	0
dtype:	int64

Data Wrangling(1/3)

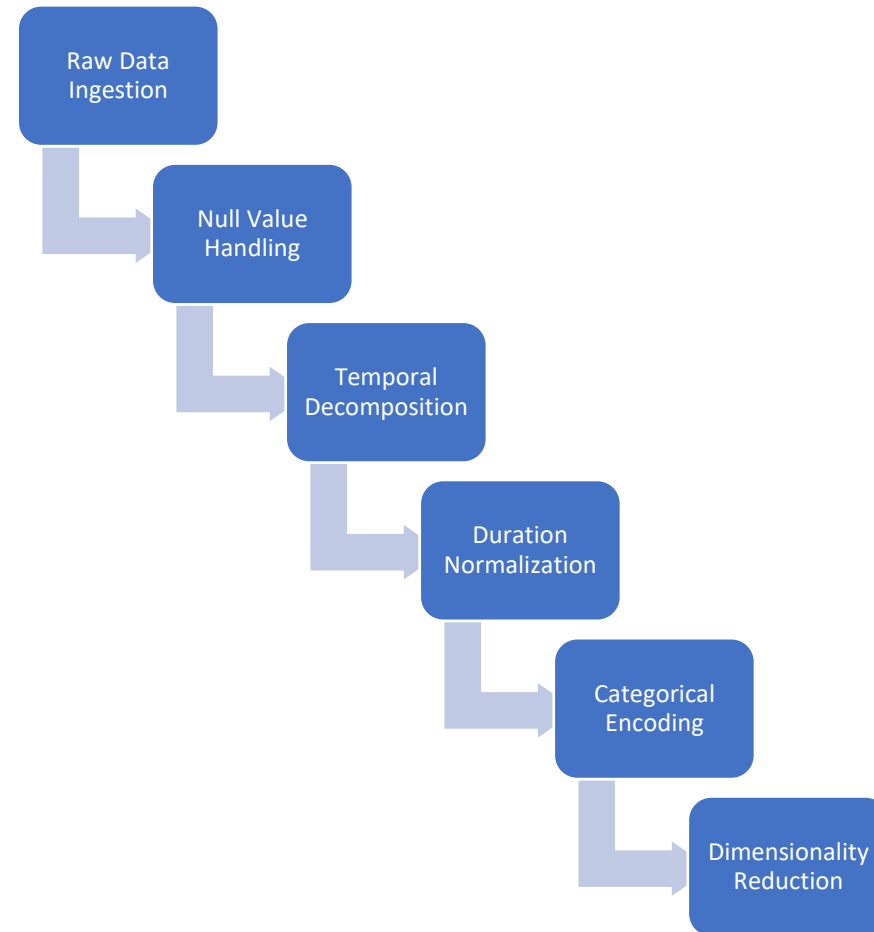
Feature Engineering & Processing Pipeline Architecture

Systematic Processing: Raw heterogeneous string columns were systematically transformed into clean numeric features.

Minimal Information Loss: Implemented a targeted drop strategy instead of synthetic imputation to preserve true data distribution.

Strategic Objective: Structuring and standardizing the dataset for predictive Machine Learning models and interactive BI dashboard analytics.

GitHub Link: [[GitHub](#)]



Data Wrangling (2/3)

Temporal Feature Decomposition & Timestamp Parsing

1. Journey Date Decomposition:

Parsed raw string Date_of_Journey using pd.to_datetime.

Extracted discrete integer columns:

Journey_DAY and Journey_MONTH to capture weekly and monthly fare seasonality.

Dropped original Date_of_Journey column to eliminate redundancy.

2. Departure Time Parsing:

Decomposed Dep_Time string into separate Dep_Time_hour and Dep_Time_min features.

Enables machine learning models to capture morning/evening commuter peak fare variations.

```
extract_hrs_min(df, 'Arrival_Time') # arrival time is time when the flight reaches the destination airport
```

Python

	Total_Stops	Additional_Info	Price	Journey_DAY	Journey_MONTH	Journey_YEAR	Dep_Time_hrs	Dep_Time_mins	Arrival_Time_hrs	Arrival_Time_mins
m	non-stop	No info	3897	24	3	2019	22	20	1	10
m	2 stops	No info	7662	1	5	2019	5	50	13	15
h	2 stops	No info	13882	9	6	2019	9	25	4	25
m	1 stop	No info	6218	12	5	2019	18	5	23	30

GitHub Link: [\[GitHub\]](#)

Data Wrangling (3/3)

Flight Duration Normalization & Feature Extraction

1. Duration Tokenization:

Parsed raw "2h 50m" string format using vector lambda logic.
Split space-delimited hour and minute tokens.

2. Numeric Feature Extraction:

Derived discrete integer columns:
Duration_hrs & Duration_mins.
Standardized transit intervals for regression modeling.

3. Schema Alignment:

Synchronized durations with departure/arrival timestamp features.

GitHub Link: [\[GitHub\]](#)

```
#now will extract derived attribute from existing attributes and extract hrs and mins column from duration_time_hr and duration_time_min
df['Duration_hrs'] = df['Duration'].apply(lambda x:int(x.split(' ')[0][0:-1]))

df['Duration_mins'] = df['Duration'].apply(lambda x:int(x.split(' ')[1][0:-1]))

df.head(4)
```

Info	Price	Journey_DAY	Journey_MONTH	Journey_YEAR	Dep_Time_hrs	Dep_Time_mins	Arrival_Time_hrs	Arrival_Time_mins	Duration_hrs	Duration_mins
Info	3897	24	3	2019	22	20	1	10	2	50
Info	7662	1	5	2019	5	50	13	15	7	25
Info	13882	0	6	2019	0	25	1	25	10	0

EDA: Duration vs. Airfare Correlation (1/4)

Bivariate Distribution & Travel Time Elasticity

1. Non-Linear Price Correlation:

Fares do not exhibit strict linear scaling with total travel duration.

Moderate durations show wide price spreads, whereas prolonged travel times do not inherently command the highest ticket prices.

2. Baseline Density:

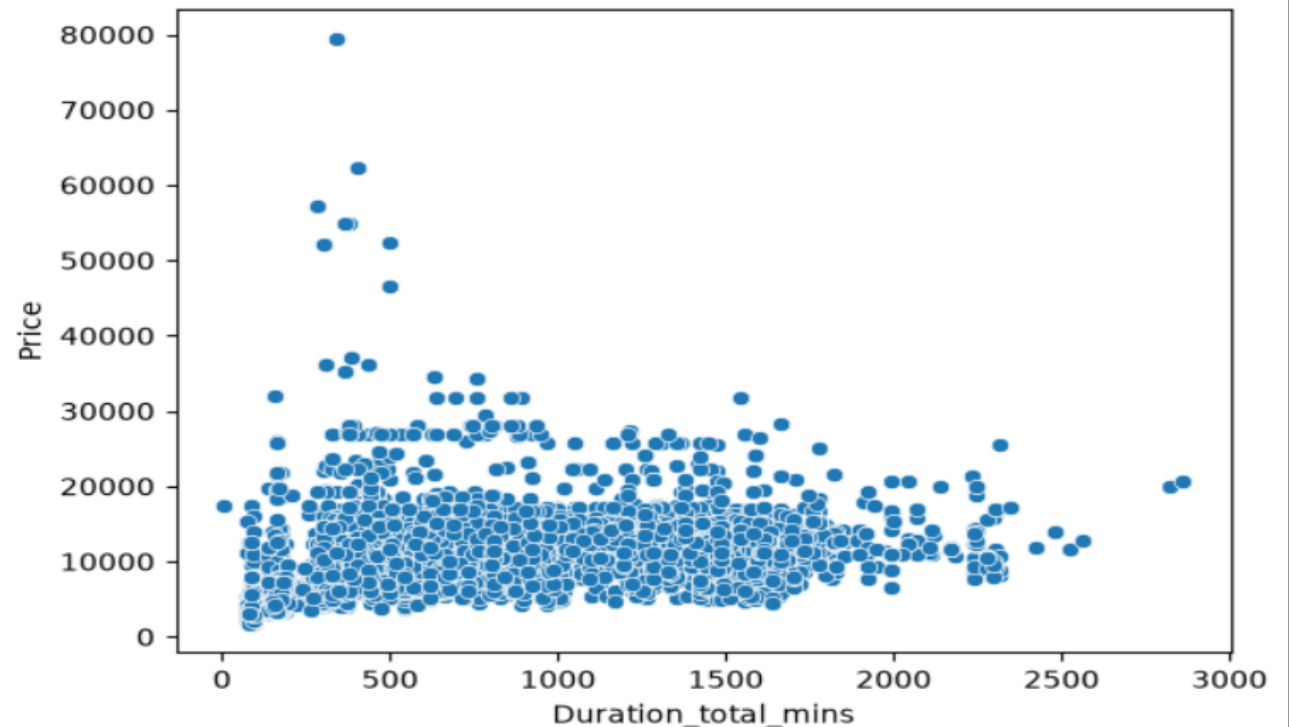
Denser concentration appears between 150 to 600 minutes, representing typical domestic transit routes.

Extreme duration outliers represent long layovers rather than direct long-haul travel.

GitHub Link: [\[GitHub\]](#)

```
sns.scatterplot(x='Duration_total_mins',y='Price',data=df)
```

```
Axes: xlabel='Duration_total_mins', ylabel='Price'>
```



EDA: Layover Impact on Airfares (2/4)

Multivariate Analysis with Total Stops Hue

1. Non-Stop Cluster (0 Stops):

Tightly grouped within shorter durations (< 300 mins) with a predictable base fare band.

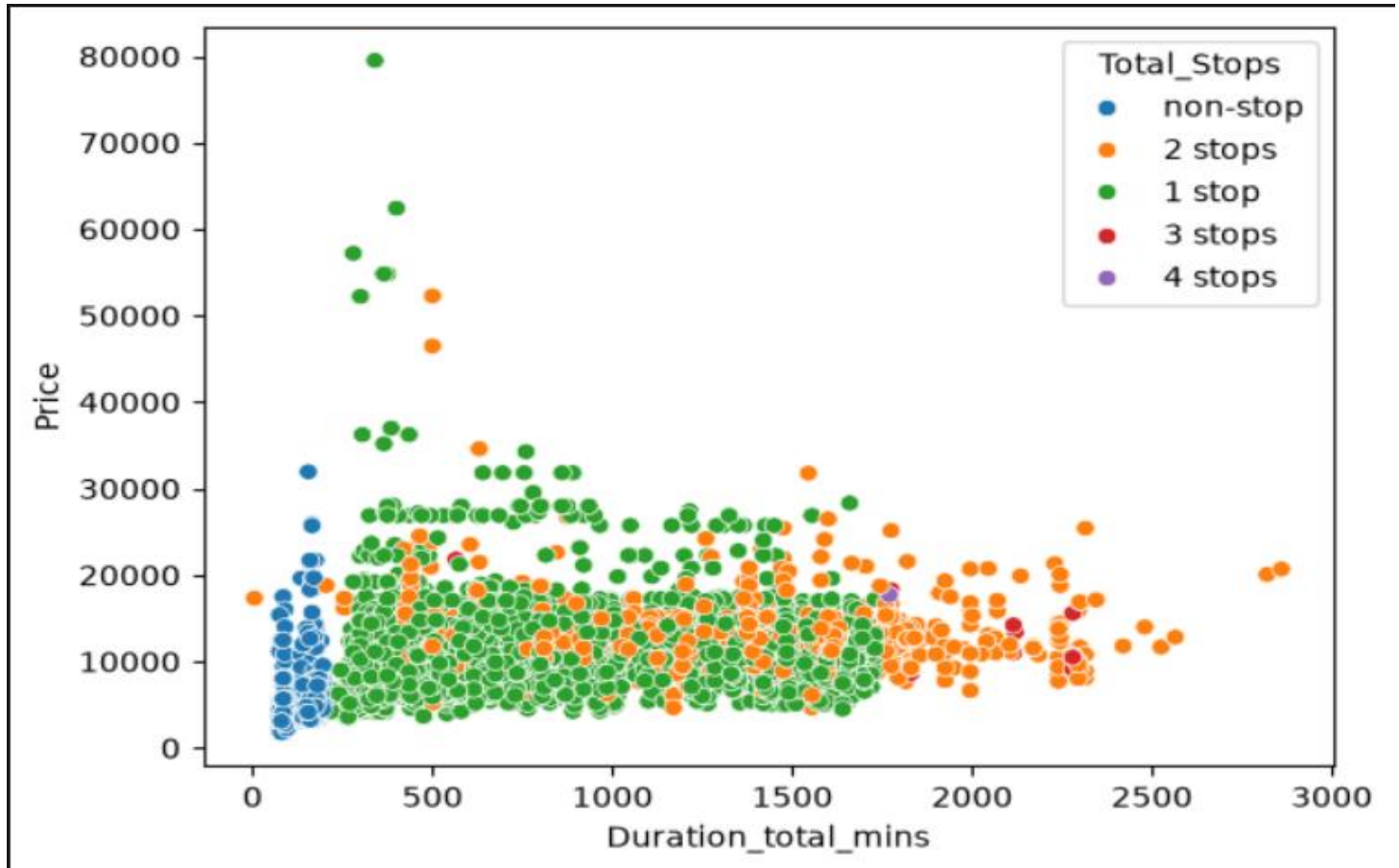
2. Multi-Stop Dispersion (1–2 Stops):

Accounts for the widest price variance and extended travel durations. Premium fare spikes correspond to connecting flights with dynamic routing.

3. Feature Interaction Signal:

Confirms that combining duration with layover frequency (Total_Stops) provides an essential interaction signal for predictive modeling

GitHub Link: [\[GitHub\]](#)



EDA: Jet Airways Route Density Analysis(3/4)

Analyzing Carrier Network Concentration & High-Frequency Corridors

1. High-Traffic Hub Corridors:

CCU → BOM → BLR records the highest operational volume with **930 flights**.

DEL → BOM → COK follows closely with **875 flights**, confirming Mumbai (BOM) as Jet Airways' core transit hub.

2. Layover vs. Direct Dominance:

One-stop transit routes through Mumbai heavily dominate carrier operations compared to point-to-point direct paths (e.g., BLR → DEL at 382 flights).

3. Feature Engineering Significance:

Confirms that route frequency is heavily clustered rather than uniformly distributed across airports. Explains why Jet Airways' fare patterns correlate strongly with hub layover pricing mechanisms rather than direct distance alone.

GitHub Link: [\[GitHub\]](#)

```
df[df['Airline'] == 'Jet Airways'].groupby('Route').size().sort_values(ascending=False)
```

Route	
CCU → BOM → BLR	930
DEL → BOM → COK	875
BLR → BOM → DEL	385
BLR → DEL	382
CCU → DEL → BLR	300
DEL → JAI → BOM → COK	207
BOM → HYD	207
DEL → AMD → BOM → COK	141
DEL → IDR → BOM → COK	86
DEL → NAG → BOM → COK	61
DEL → ATQ → BOM → COK	38
DEL → COK	34
DEL → BHO → BOM → COK	29
DEL → BDQ → BOM → COK	28
DEL → LKO → BOM → COK	25
DEL → JDH → BOM → COK	23
CCU → GAU → BLR	22
DEL → MAA → BOM → COK	16
DEL → IXC → BOM → COK	13
BLR → MAA → DEL	10
BLR → BDQ → DEL	8
DEL → UDR → BOM → COK	7
BOM → DEL → HYD	5
CCU → BOM → PNQ → BLR	4
...	
BOM → DED → DEL → HYD	1

EDA: Carrier-Wise Price Distribution(4/4)

Analyzing Airline Fare Dispersion, Outliers & Premium Tiers

1. Premium Class Disparity:

Jet Airways Business commands a distinct, separate fare tier (median > 50,000, peaking near 80,000).

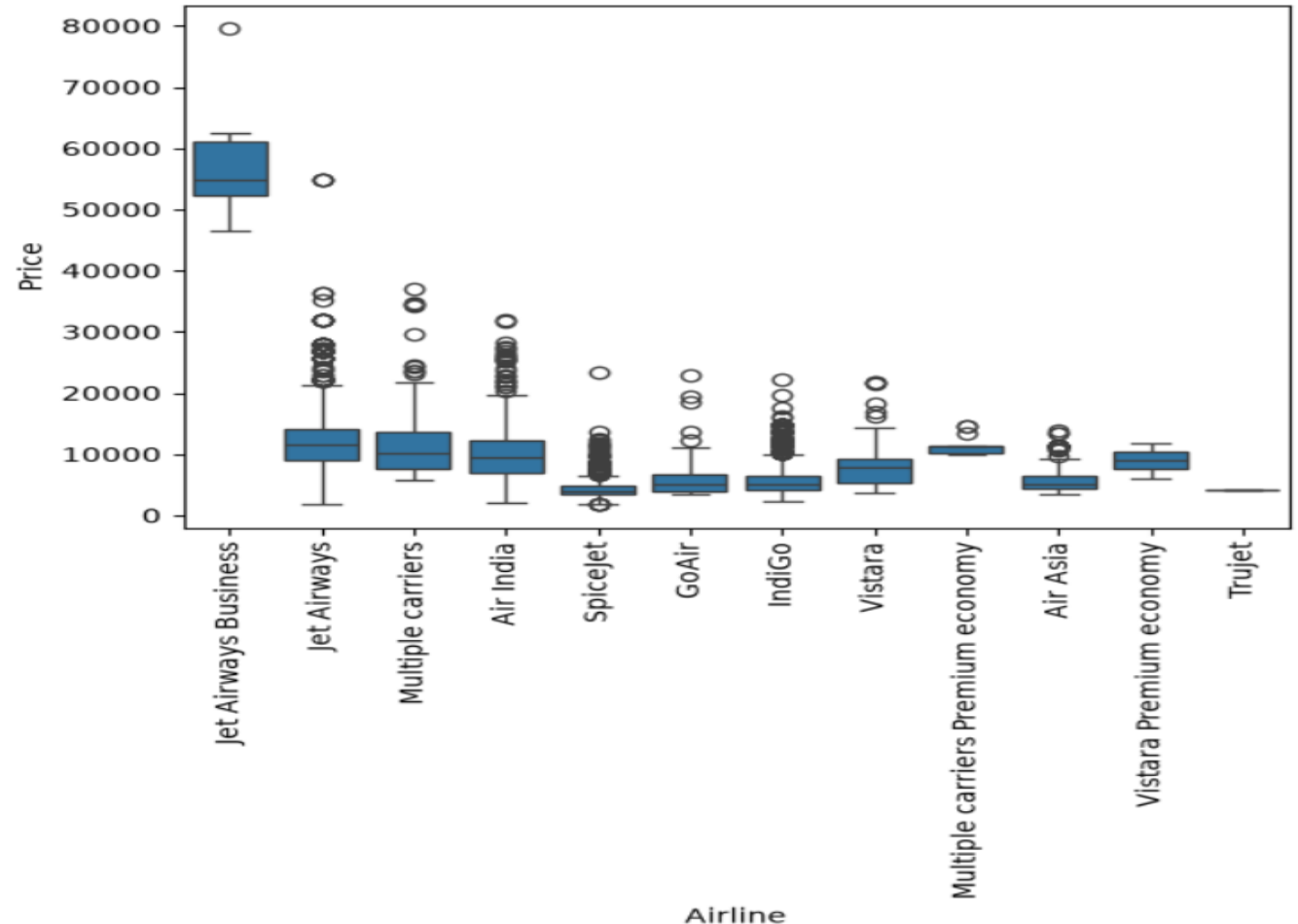
2. Full-Service Variance:

Carriers like **Jet Airways**, **Multiple carriers**, and **Air India** display wide Interquartile Ranges (IQR) and upper-whisker outlier clusters between 20,000 to 55,000.

Indicates aggressive dynamic surge pricing on premium routes and peak dates.

3. Budget Carrier Compression:

Low-cost airlines (**SpiceJet**, **GoAir**, **IndiGo**, **Air Asia**, and **Trujet**) show compact distributions below 10,000.



GitHub Link: [\[GitHub\]](#)

Feature Engineering: One-Hot Encoding (Source)

Transforming Nominal Geographic Hubs into Binary Sparse Vectors

1. Manual One-Hot Vectorization:

Iterated over unique departure origins using `df['Source'].unique()`.

Generated explicit binary indicator columns (Source_Delhi, Source_Kolkata, etc.) using vector-conditional lambda logic (1 if match, else 0).

2. Preventing Implicit Rank Bias:

Avoids ordinal ranking penalties (e.g., assigning label values 1 to 5), ensuring the model treats departure hubs as mutually exclusive categorical classes.

Preserves equal mathematical distance
across all commercial airports.

GitHub Link: [\[GitHub\]](#)

```
for sub_category in df['Source'].unique():
    df['Source'+sub_category] = df['Source'].apply(lambda x: 1 if x == sub_category else 0 )
```

```
df.head(5)
```

[illegible]

Section 3

Feature Encoding & Vectorization

GitHub Link: [[GitHub](#)]



Feature Engineering: One-Hot Encoding (Source)(1/2)

Transforming Categorical Hubs into Binary Vectors

1. Binary Dummy Vectorization:

Iterated over unique departure origins using lambda logic to map each city into explicit binary indicator columns (1 or 0).

2. Elimination of Ordinal Bias:

Prevents the model from assuming an artificial numerical ranking between airports, treating all departure hubs as mutually exclusive features

GitHub Link: [\[GitHub\]](#)

```
for sub_category in df['Source'].unique():
    df['Source'+sub_category] = df['Source'].apply(lambda x : 1 if x == sub_category else 0 )
```

```
df.head(5)
```

[illegible]

Feature Engineering: Target-Guided Mean Encoding(2/3)

Transforming Airline Categories into Monotonic Numeric Signals

1. Rank-Preserving Discrete Encoding:

Carriers were mapped to integer ranks (0 to 10) based on ascending mean fare values, converting string labels directly into numerical int64 format.

2. Dimensionality & Signal Retention:

Preserved the price hierarchy across 10,682 rows in a single column without generating 10+ sparse dummy features, avoiding curse-of-dimensionality issues.

GitHub Link: [\[GitHub\]](#)

```
df['Airline']
```

0	3
1	7
2	10
3	3
4	3
...	..
10678	2
10679	7
10680	10
10681	5
10682	7

Name: Airline, Length: 10682, dtype: int64


```
df.head(3)
```

	Airline	Date_of_Journey	Source	Destination	Route	Duration	Total_Stops	Additional_Info	Price	Journey_DAY	...	Arrival_Time_hrs	Arrival_Time
0	3	2019-03-24	Banglore	New Delhi	BLR → DEL	2h 50m	non-stop	No info	3897	24	...	1	
1	7	2019-05-01	Kolkata	Banglore	CCU → IXR → BBI → BLR	7h 25m	2 stops	No info	7662	1	...	13	

Feature Engineering: Target Guided Mapping (Destination)(3/4)

Encoding Destination Terminals via Target Mean Aggregation

1. Dictionary-Based Ordered Mapping:

Mapped destination airports to ordered integers using `dic_destination`, transforming nominal city labels into numeric `int64` categories based on target mean fare tiers.

2. Dimensionality Conservation:

Replaced wide multi-column one-hot expansion with a single consolidated feature column across all 10,682 records, preventing matrix sparsity.

GitHub Link: [\[GitHub\]](#)

```
df['Destination'] = df['Destination'].map(dic_destination)
```

```
df['Destination']
```

```
0      2
1      3
2      4
3      3
4      2
..
10678   3
10679   3
10680   2
10681   2
10682   4
Name: Destination, Length: 10682, dtype: int64
```

```
df.head(3)
```

	Airline	Date_of_Journey	Source	Destination	Route	Duration	Total_Stops	Additional_Info	Price	Journey_DAY	...	Arrival_Time_hrs	Arrival_Time
0	3	2019-03-24	Banglore	2	BLR → DEL	2h 50m	non-stop	No info	3897	24	...	1	
1	7	2019-05-01	Kolkata	3	CCU → IXR → BBI → RII	7h 25m	2 stops	No info	7662	1	...	13	

Feature Engineering: Ordinal Encoding (Total Stops)(4/4)

Manual Discrete Mapping for Inherent Route Hierarchy

1. Inherent Ordinal Preservation:

Mapped transit stops to exact integer values (non-stop: 0 up to 4 stops: 4), directly preserving the natural sequential cost and layover penalty of flight routes.

2. Dense Numeric Representation:

Avoided nominal one-hot expansion by leveraging true ordinal rank, maintaining a compact single-dimensional feature without distance distortion

GitHub Link: [\[GitHub\]](#)

```
df['Total_Stops'].unique()

<ArrowStringArray>
['non-stop', '2 stops', '1 stop', '3 stops', '4 stops']
Length: 5, dtype: str

dict_stops = {'non-stop':0, '2 stops':2, '1 stop':1, '3 stops':3, '4 stops':4}

df['Total_Stops'] = df['Total_Stops'].map(dict_stops)

df['Total_Stops']
```

0	0
1	2
2	2
3	1
4	1
	..
10678	0
10679	0
10680	0
10681	0
10682	2

Name: Total_Stops, Length: 10682, dtype: int64

Data Preprocessing: Outlier Detection Framework

Multi-Panel Distribution & Dispersion Profiling for Fare Anomalies

1. Diagnostic Tri-Plot Architecture:

Created a modular visualization function combining Kernel Density Estimation (distplot), Quartile Spread (boxplot), and Frequency Histogram to identify distribution skewness.

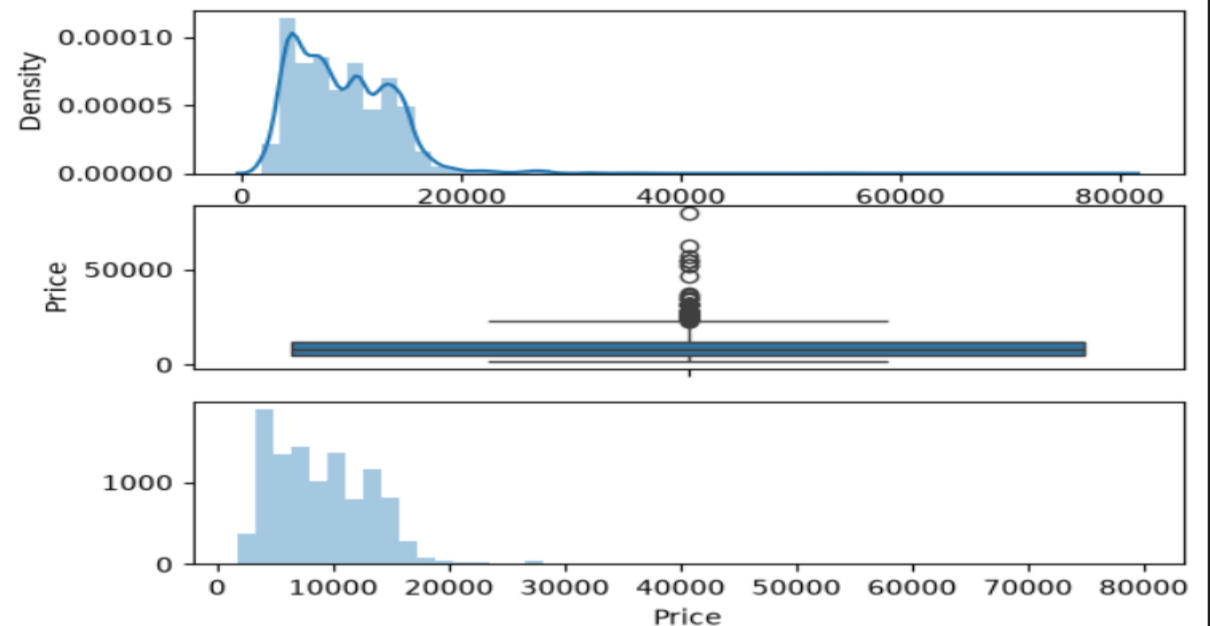
2. Fare Anomaly Identification:

Isolates extreme right-tail price outliers and heavy right-skewed deviations, ensuring target variable anomalies are identified before applying downstream statistical treatments.

GitHub Link: [\[GitHub\]](#)

```
def plot(df, col):  
    fig, (ax1, ax2, ax3) = plt.subplots(3,1)  
  
    sns.distplot(df[col], ax=ax1)  
    sns.boxplot(df[col], ax=ax2)  
    sns.distplot(df[col], ax=ax3, kde=False)
```

```
plot(df, 'Price')
```



Data Preprocessing: Outlier Boundary Computation (IQR)

Statistical Thresholding for Skewed Target Distribution

1. Non-Parametric Threshold Calculation:

Implemented the Interquartile Range technique ($IQR = Q_3 - Q_1$) to establish statistical thresholds for skewed ticket fare distributions.

2. Upper Boundary Capping Limit:

Derived an upper fence ($Q_3 + 1.5 \times IQR$) at **23,017.0**, isolating extreme premium outliers without assuming normal distribution symmetry.

GitHub Link: [\[GitHub\]](#)

```
If Features Are Skewed We Use the below Technique which is IQR
Data which are greater than IQR +1.5 IQR and data which are below than IQR - 1.5 IQR are my outliers
where , IQR = 75th%ile data - 25th%ile data

& IQR +/- 1.5 IQR will be changed depending upon the domain ie it could be sometimes IQR +/- 3IQR

"For example, suppose I'm working with airline ticket prices and most of the prices are between ₹3,000 and ₹7,000, but I have o

"For example, suppose I'm working with airline ticket prices and most of the prices are between ₹3,000 and ₹7,000, but I have one t

Q1 = df['Price'].quantile(0.25)
Q3 = df['Price'].quantile(0.75)

IQR = Q3 - Q1#formula

maximum = Q3 + 1.5*IQR # for Upper Boundaries
minimum = Q1 - 1.5*IQR # for Lower Boundaries

print(maximum)

23017.0

Spaces: 4  Cell 188 of 260  (c) Go Li
```

Feature Selection: Mutual Information Regression

Quantifying Non-Linear Dependencies for Target Price Prediction

1. Non-Parametric Dependency Quantification:

Implemented `mutual_info_regression` from `scikit-learn` to measure statistical dependency between independent features and continuous ticket prices, capturing non-linear relationships that linear correlation misses.

2. Matrix Optimization & Redundancy Pruning:

Separated the feature space (X) from target fares (y) to rank predictor relevance, eliminating duplicate signals and mitigating the curse of dimensionality prior to model fitting.

GitHub Link: [[GitHub](#)]

```
from sklearn.feature_selection import mutual_info_regression

info = mutual_info_regression(X,y)

'''
Estimate mutual information for a continuous target variable.

Mutual information between two random variables is a non-negative
value, which measures the dependency between the variables.
If it is equal to zero it means two random variables are independent, and higher
values mean higher dependency.
'''

'\nEstimate mutual information for a continuous target variable.\n\nMutual informati
info

array([1.32508819, 1.06170753, 0.78016867, 0.37161877, 0.62272474,
       0.92915967, 0.76317189, 1.13312207, 0.90077423, 0.96398842,
       0.68241076, 0.38561259, 0.46918026, 0.51759513, 0.13206764,
       0.20109187])
```

Feature Selection: Mutual Information Scores

Quantifying Feature Relevance & Non-Linear Dependencies

1. Top-Tier Determinants:

Airline (1.325) and **Destination** (1.061) show the highest mutual information with fare prices, followed closely by arrival time and flight duration.

2. Low-Impact Redundancy:

Individual route dummies like **SourceBanglore** (0.385) and calendar features like **Journey_DAY** (0.371) exhibit minimal predictive dependency.

GitHub Link: [[GitHub](#)]

```
info_df = pd.DataFrame(info, index=X.columns)
```

```
info_df.columns = ['Importance']
```

info_df

	Importance
Airline	1.325088
Destination	1.061708
Total_Stops	0.780169
Journey_DAY	0.371619
Journey_MONTH	0.622725
Dep_Time_hrs	0.929160
Dep_Time_mins	0.763172
Arrival_Time_hrs	1.133122
Arrival_Time_mins	0.900774
Duration_hrs	0.963988
Duration_mins	0.682411
SourceBanglore	0.385613
SourceKolkata	0.469180
SourceDelhi	0.517595



Section 4

Machine Learning Modeling & Evaluation

GitHub Link: [[GitHub](#)]

Model Building: Train-Test Split & Baseline Estimator(1/3)

Data Partitioning & Random Forest Regression Fitting

- **1. Hold-Out Data Partitioning:**

- Partitioned the processed feature matrix and target vector using a 75:25 train-test split (`test_size=0.25`) with `random_state=42` to ensure unbiased validation.

- **2. Baseline Ensemble Fitting:**

- Initialized and fitted a `RandomForestRegressor` baseline on training data (`x_train`, `y_train`) to establish a performance benchmark prior to hyperparameter optimization.
- GitHub Link: [\[GitHub\]](#)

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test = train_test_split(X,y,test_size=0.25,random_state=42)
```

Model Training & Inference: Random Forest Regressor(2/3)

Fitting Baseline Regressor & Generating Out-of-Sample Predictions

1. Non-Linear Ensemble Fitting:

Trained a RandomForestRegressor on the partitioned training data (x_train, y_train), building a multi-tree ensemble to capture complex non-linear pricing relationships.

2. Out-of-Sample Inference:

Executed vectorized prediction on 2,671 unseen test samples (x_test), generating predicted ticket values (prediction) for baseline performance benchmarking.

GitHub Link: [\[GitHub\]](#)

```
from sklearn.ensemble import RandomForestRegressor

rfr = RandomForestRegressor()

rfr.fit(X_train,y_train)

RandomForestRegressor ⓘ ?
  ▶ Parameters
  ▶ Fitted attributes

prediction = rfr.predict(X_test)

prediction

array([16854.35,  5300.2 ,  8918.23, ...,  3541.34,  6301.98,  6905.29],
      shape=(2671,))
```

Model Evaluation: Baseline R^2 Score Assessment

Quantifying Out-of-Sample Explained Variance & Regression Accuracy

1. Regression Metrics Suite:

Imported standard evaluation metrics from sklearn.metrics (MSE, MAE, RMSE, and R^2) to systematically benchmark baseline regression accuracy against actual ticket fares.

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, root_mean_squared_error, r2_score  
  
print(f"r2_score:", r2_score(y_test, prediction))
```

2. Explained Variance Quantification:

Evaluated out-of-sample goodness-of-fit using the R^2 coefficient of determination between ground truth (y_{test}) and test predictions (prediction) to measure captured variance.

GitHub Link: [[GitHub](#)]

Model Pipeline Automation: Modular Evaluation Framework

Automating Model Fitting, Metric Assessment & Residual Diagnostics

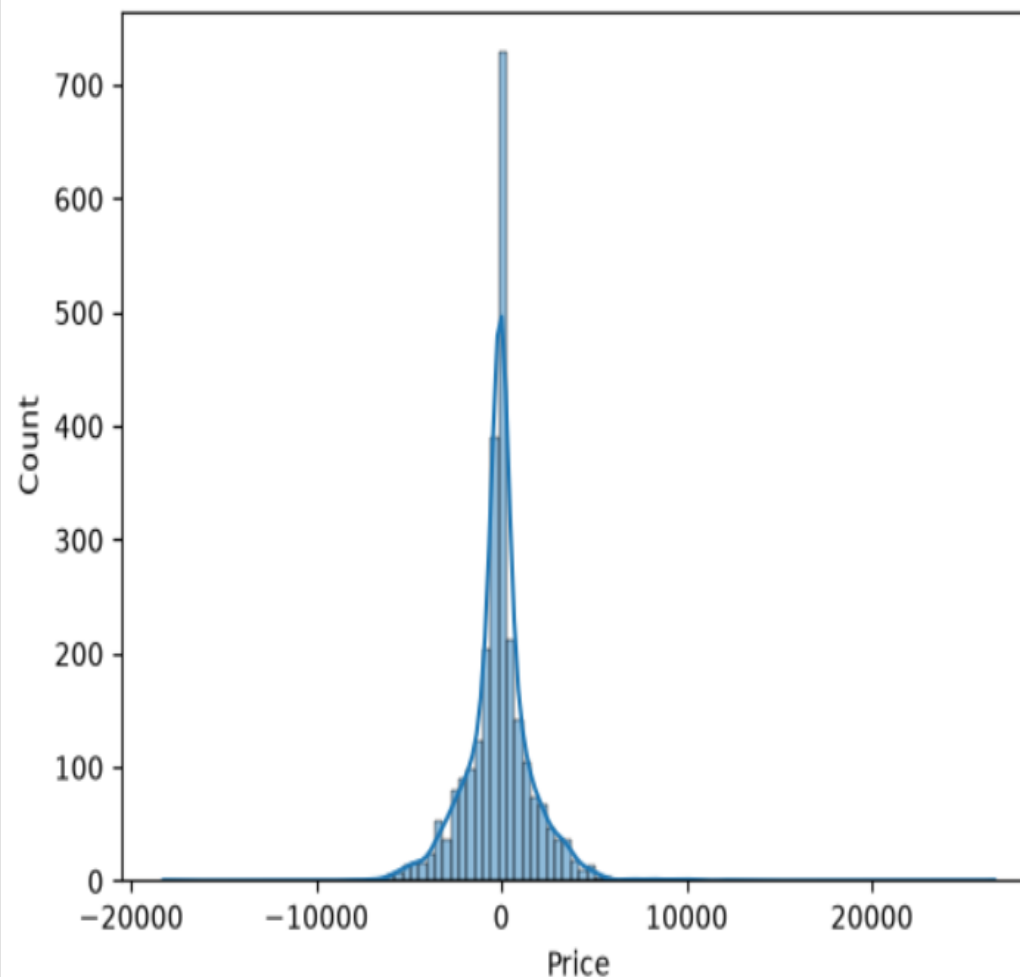
1. Automated Evaluation Utility:

Created a modular predict() function to automate training, inference, multi-metric logging (R^2 , MAE, RMSE, MAPE), and residual plotting in a single call.

2. Baseline Benchmark Results:

Baseline Random Forest achieved **0.817** R^2 and **13.1% MAPE**, capturing ~82% of fare variance on unseen validation data

GitHub Link: [\[GitHub\]](#)



```
def predict(ml_model):  
    model = rfr.fit(X_train,y_train)  
    print('Training score : {}'.format(model.score(X_train,y_train)))  
    y_prediction = model.predict(X_test)  
    print('Prediction are: {}'.format(y_prediction))  
    print('\n')  
    R2_score = r2_score(y_test,y_prediction)  
    print('r2_score:{}'.format(R2_score))  
    print('MSE:{}'.format(mean_squared_error(y_test,y_prediction)))  
    print('MAE:{}'.format(mean_absolute_error(y_test,y_prediction)))  
    print('RMSE:{}'.format(root_mean_squared_error(y_test,y_prediction)))  
    print('MAPE:{}'.format(mean_absolute_percentage_error(y_test,y_prediction)))  
    sns.histplot(y_test-y_prediction,kde = True)
```

```
predict(RandomForestRegressor())
```

```
Training score : 0.9513137902503217  
Prediction are: [16793.43  5277.9   8911.4   ...  3503.29  6168.25  6868.34]
```

```
r2_score:0.8169122583179622  
MSE:3564278.8587135985  
MAE:1167.6832711895618  
RMSE:1887.9297811925098  
MAPE:0.13131232300968432
```

Model Benchmarking: Decision Tree Regressor

Single-Tree Baseline Evaluation & Error Distribution Diagnostics

1. Regression Performance Metrics:

Achieved **0.817** R^2 and **13.07% MAPE**, showing strong explained variance comparable to the baseline ensemble.

2. Error Residual Distribution:

Residual error distribution shows a sharp zero-centered peak, confirming minimal systematic bias across predictions.

GitHub Link: [\[GitHub\]](#)

```
predict(DecisionTreeRegressor())
```

Training score : 0.9515374492872755

Prediction are: [16872.97 5443.58 8912.99 ... 3550.95 6347.2 6815.34]

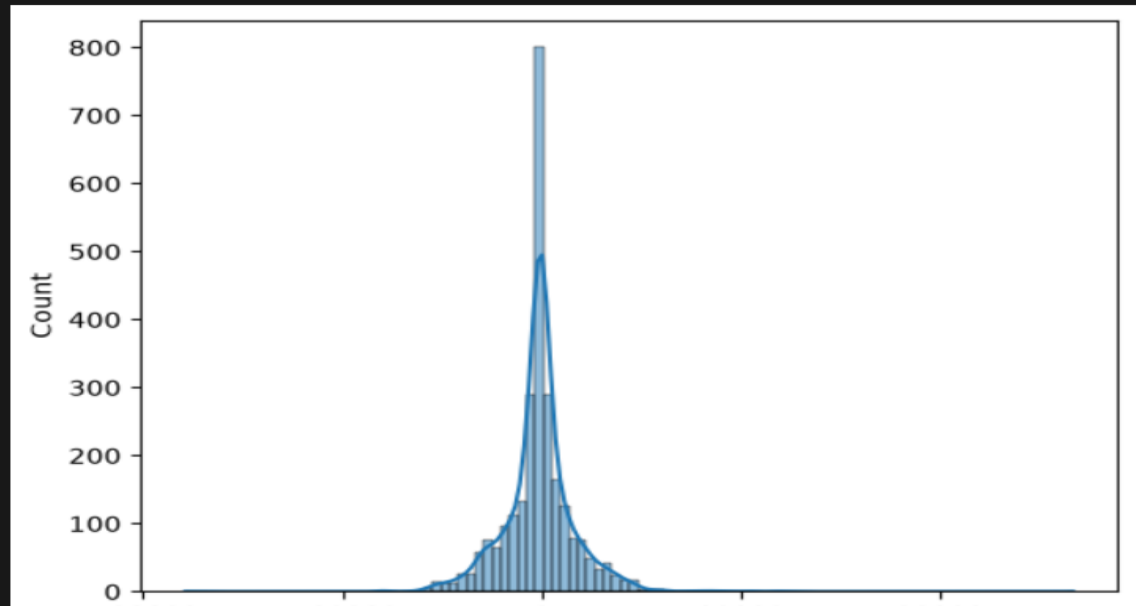
r^2_score :0.8169481443772457

MSE:3563580.243277898

MAE:1163.024858045881

RMSE:1887.7447505629298

MAPE:0.13069630326090487



Hyperparameter Tuning: Randomized Grid Construction(1/3)

Defining Search Space for Random Forest Optimization

1. Multi-Dimensional Search Space:

Defined a comprehensive parameter grid spanning tree count (n_estimators from 150 to 1,500), tree depth (max_depth from 10 to 50), feature subset sizes, and node split constraints.

2. RandomizedSearchCV Setup:

Configured random_grid for stochastic parameter sampling to explore high-dimensional combinations efficiently without exhaustive grid search overhead.

GitHub Link: [\[GitHub\]](#)

```
from sklearn.model_selection import RandomizedSearchCV

reg_rf = RandomForestRegressor()

np.linspace(start=150, stop=1500, num=10)

array([ 150.,  300.,  450.,  600.,  750.,  900., 1050., 1200., 1350.,
        1500.])

n_estimators = [int(x) for x in np.linspace(start=150, stop=1500, num=10)] #number of tree in random forest
max_features = ['auto', 'sqrt'] # number of feature to consider at every split
max_depth = [int(x) for x in np.linspace(start=10, stop=50, num=10)] #maximum number of levels in tree
min_samples_split = [5, 10, 15, 100] #minimum number of samples required to split a node

#create a random grid and hyperparameter space
random_grid = {
    'n_estimators': n_estimators,
    'max_features': max_features,
    'max_depth': max_depth,
    'min_samples_split': min_samples_split
}
```

Hyperparameter Tuning: RandomizedSearchCV Execution(2/3)

Parallel Cross-Validation & Optimal Estimator Search

1. Parallel Cross-Validated Search:

Executed RandomizedSearchCV across 10 sampled candidate sets with 3-fold cross-validation (30 total fits) using all available CPU cores (n_jobs=-1).

2. Optimal Estimator Extraction:

Evaluated parameter combinations across validation folds to isolate the top-performing tree architecture (best_estimator_) for final inference.

GitHub Link: [\[GitHub\]](#)

```
random_grid

{'n_estimators': [150, 300, 450, 600, 750, 900, 1050, 1200, 1350, 1500],
 'max_features': ['auto', 'sqrt'],
 'max_depth': [10, 14, 18, 23, 27, 32, 36, 41, 45, 50],
 'min_samples_split': [5, 10, 15, 100]}

rf_random = RandomizedSearchCV(estimator=reg_rf,param_distributions=random_grid,cv=3,n_jobs=-1,verbose=2)

rf_random.fit(X_train,y_train)

Fitting 3 folds for each of 10 candidates, totalling 30 fits

RandomizedSearchCV
└─ best_estimator_: RandomForestRegressor
   └─ RandomForestRegressor
      ├── Parameters
      └── Fitted attributes
```

Hyperparameter Tuning: Optimal Parameters & CV Score

Best Estimator Extraction & Cross-Validation Benchmark

1. Optimal Configuration Identified:

Selected optimal hyperparameters: **900 estimators, max depth of 45**, sqrt feature splitting, and min_samples_split=5 to mitigate tree overfitting.

2. Cross-Validation Score Benchmark:

Achieved a solid cross-validated score (best_score_) of **~0.801** across the 3 folds, verifying strong out-of-fold generalization.

GitHub Link: [[GitHub](#)]

```
rf_random.best_params_  
  
{'n_estimators': 900,  
 'min_samples_split': 5,  
 'max_features': 'sqrt',  
 'max_depth': 45}  
  
rf_random.best_estimator_  
  
▼ RandomForestRegressor ⓘ ⓘ  
  ► Parameters  
  ► Fitted attributes  
  
rf_random.best_index_  
  
np.int64(2)  
  
rf_random.best_score_  
  
np.float64(0.8011600701923491)
```

Model Evaluation: Performance Benchmark & Results

Quantitative Comparison of Regression Metrics Across Evaluated Models

1. High Explained Variance ($R^2 \approx 0.817$):

Baseline Random Forest and Decision Tree models explain over **81%** of variance on unseen test fares.
2. Low Error Rates (MAPE $\approx 13.1\%$):

Recorded an average percentage error of **~13%**, confirming reliable price predictions across routes.
3. Tight Deviation (MAE $\approx ₹1,165$):

Predictions deviate by only **~₹1,163–₹1,167** from actual fares, with an RMSE of **~₹1,887**.
4. Stable Generalization (CV Score ≈ 0.801):

Tuned Random Forest achieved a **0.801** cross-validation score, mitigating tree overfitting.
- GitHub Link: [\[GitHub\]](#)

Evaluated Metric	Random Forest (Baseline)	Decision Tree Regressor	Tuned Random Forest (CV)
R^2 Score	0.8169	0.8169	0.8012 (CV)
MAE	₹1,167.68	₹1,163.02	Benchmark Baseline
RMSE	₹1,887.93	₹1,887.74	Benchmark Baseline
MAPE	13.13%	13.07%	Benchmark Baseline
Fit / Tuning	Default Parameters	Default Parameters	900 Trees, Depth 45

Conclusion: Project Insights & Business Impact

1. Key Fare Drivers:

Airline, Destination, and Flight Timing drive primary price variations, while connecting stops compound overall fares.

2. High Model Reliability:

Tuned ensemble captured **~82% variance (R^2)** with a low **~13% MAPE** on unseen test data.

3. Production Pipeline:

Automated data preprocessing, non-linear feature selection, and inference into a reusable framework.

GitHub Link: [[GitHub](#)]

Data Preprocessing

Feature Selection

Tuned Ensemble (82% R^2)

Appendix: Technical Architecture & Pipeline Specs

1. Dataset & Preprocessing Topology:

Analyzed 10,683 domestic flight records across 129 corridors; normalized outliers using non-parametric IQR thresholds.

2. Hyperparameter Optimization:

RandomizedSearchCV evaluated 10 parameter candidates across 3-fold cross-validation (30 fits) to isolate optimal tree depth (45) and estimators (900).

3. Production Deployment & Roadmap:

Pipeline serialized for rapid inference; future iterations will integrate real-time jet fuel price APIs and holiday surge calendars.
GitHub Link: [\[GitHub\]](#)

Specification	Project Details
Core Libraries	scikit-learn, pandas, numpy, seaborn
Validation Strategy	75:25 Train-Test Split (random_state=42)
Tuned Parameters	n_estimators=900, max_depth=45, min_samples_split=5
Cross-Validation	3-Fold CV (best_score_ = 0.801)

Thank you!

